



11/02/2025

Reverse Engineering and Refactoring of Rudra's Subscription Service (RSS)

By Team - 17

Priyank N

name@email.address

Aditya Singh Rathore

name@email.address

Yash Sonkar

2024801002

name@email.address

Sanket Adlak

name@email.address

Nitheesh Chandra

2024801002

nitheeshchandra.y@research.iiit.ac.in

Faculty:

Karthik Vaidhyanathan

Semester:

Spring 2025

Course:

CS6.401 Software Engineering

Contents

- LIST OF FIGURES II
- LIST OF TABLES III
- 1. INTRODUCTION 1
- 2. PROJECT OBJECTIVES 1
- 3. SYSTEM OVERVIEW 1
- 4. TASK 1: REVERSE ENGINEERING 1
 - 4.1. Identification of Relevant Classes 2
 - 4.2. UML Class Diagram 2
 - 4.3. Observation and Comments 2
- 5. TASK 2: DESIGN SMELLS AND CODE METRICS 2
 - 5.1. Design Smell Analysis 2
 - 5.2. Code Metrics 2
- 6. TASK 3: REFACTORING 3
 - 6.1. Addressing Design Smells 3
 - 6.1.1. Refactoring Strategy 3
 - 6.2. Code Metrics Post Refactoring 4
 - 6.3. Leveraging LLMs for Refactoring 4
 - 6.4. Automation of Refactoring Pipeline 4
- 7. CONCLUSION 4
- BIBLIOGRAPHY 4
- 8. ACKNOWLEDGEMENTS 4
- APPENDICES 5
 - Appendix A: AI Prompt Screenshots 5
 - Appendix B: Additional UML Diagrams 5

List of Figures

Figure 1: A curious figure. 2

Figure 2: Example 1 3

List of Tables

Table 1: Design Smells Identified in the RSS Reader	2
Table 2: Code Metrics Extracted from the RSS Reader	3
Table 3: Code Metrics Post Refactoring	4

1. Introduction

The goal of this project is to analyze, document, and improve the existing Rudra's Subscription Service (RSS) Reader. The project involves reverse engineering the current codebase, identifying design smells and code metrics, and implementing refactoring strategies. The project aims to improve the maintainability, readability, and performance of the RSS Reader. **TODO: Still to be finalized**

2. Project Objectives

- Analyze the existing RSS Reader codebase.
- Identify and document relevant system classes and components.
- Detect and justify design smells.
- Measure software quality through code metrics.
- Refactor the system based on software engineering principles.
- Compare manual refactoring with AI-generated suggestions.
- Automate the refactoring pipeline.

3. System Overview

Rudra's Subscription Service (RSS) Reader is a web-based RSS aggregator that allows users to subscribe to and organize RSS feeds. The major subsystems include:

1. Subscription and Content Subsystem - Handles feed subscriptions, imports/exports, and displays recent articles.
2. Feed Organization Subsystem - Allows users to manage, categorize, and search for articles.
3. User Management Subsystem - Manages user accounts and authentication.

4. Task 1: Reverse Engineering

TODO: Details to be added to below sections

4.1. Identification of Relevant Classes

4.2. UML Class Diagram



Figure 1: **A curious figure.**

4.3. Observation and Comments

As per Figure 1 we can observe that ...

5. Task 2: Design Smells and Code Metrics

5.1. Design Smell Analysis

Design Smell	Description	Justification
Feature Envy	UPDATE: A method in "This" Class accesses another class's data more than its own.	High coupling between classes.

Table 1: **Design Smells Identified in the RSS Reader**

5.2. Code Metrics

Tools Used

- Sonarqube
- CodeMR
- Checkstyle
- PMD

Extracted Metrics

Metric	Value	Implication
Cyclomatic Complexity	5	Moderate complexity.
Code Duplication	10%	High code duplication.

Table 2: **Code Metrics Extracted from the RSS Reader**

6. Task 3: Refactoring

6.1. Addressing Design Smells

6.1.1. Refactoring Strategy

```
@@ -29,26 +51,10 @@ public class FaviconUpdateRequestedAsyncListener {
    */
    @Subscribe
    public void onFaviconUpdateRequested(final FaviconUpdateRequestedEvent
faviconUpdateRequestedEvent) throws Exception {
-     if (log.isInfoEnabled()) {
-         log.info(MessageFormat.format("Favicon update requested event:
{0}", faviconUpdateRequestedEvent.toString()));
-     }
-
+
+     logFaviconRequest(faviconUpdateRequestedEvent);
+
+ }
```

Or



Figure 2: **Example 1**

6.2. Code Metrics Post Refactoring

Metric	Value	Implication
Cyclomatic Complexity	3	Reduced complexity.
Code Duplication	5%	Reduced code duplication.

Table 3: **Code Metrics Post Refactoring**

6.3. Leveraging LLMs for Refactoring

LLM based Refactoring Analysis

- Manual Refactoring vs LLM based suggestions.
- Accuracy and efficiency of LLM suggestions.

6.4. Automation of Refactoring Pipeline

- Automated Design Smell Detection
- Github Actions Pipeline for Continuous Refactoring

7. Conclusion

...

Bibliography

8. Acknowledgements

Appendices

Appendix A: AI Prompt Screenshots

Appendix B: Additional UML Diagrams